

How Jailbreak Attacks Inform Safety Alignment: A Defender-Centric, Shapley-Based Evaluation of Jailbreak Contributions

Yukai Zhou¹, Feiyang Lu¹, Xiaokai Mao², Jinfei Liu², Wenjie Wang^{1†}

¹ShanghaiTech University, ²Zhejiang University,
{zhouyk12023, lufy2023, wangwj1}@shanghaitech.edu.cn,
{xiaokaimao, jinfeiliu}@zju.edu.cn

Abstract

Jailbreak attacks on large language models are usually evaluated by attacker-centric metrics such as attack success rate (ASR), yet an attack that breaks a model is not necessarily useful for improving its safety. We propose a defender-centric view of jailbreak evaluation, where attacks are evaluated by the downstream safety improvements they enable when used as red-teaming data for safety training. Building on this view, we introduce **A-MESS (Minimal Effective Attack-Subset Selection)**, a setting-agnostic framework for attributing and selecting jailbreak attacks from black-box subset utility observations. A-MESS estimates **AttackSHAP**, a Shapley-based score that attributes marginal utility to individual attacks and selects compact attack subsets under user-specified budgets via greedy or surrogate-based optimization. Across controlled utility landscapes and real LLM safety settings, we find that ASR rankings are weakly aligned with defender-centric utility, that AttackSHAP can be estimated accurately with limited utility queries, and that directly optimizing subsets yields stronger safety utility than attacker-centric or attribution-only selection. These results suggest evaluating jailbreak attacks as resources for improving safety, not only as tools for breaking models. ¹

1 Introduction

Jailbreak attacks have become a central tool for stress-testing the safety of large language models (LLMs) (Carlini et al., 2023; Wei et al., 2023a; Jain et al., 2023; Yi et al., 2024; Chu et al., 2025). By constructing adversarial prompts (Zou et al., 2023; Zhu et al., 2024b; Liao and Sun, 2024; Zhou et al., 2025a; Beyer et al., 2026) or interaction strategies (Chao et al., 2025; Ren et al., 2025; Andriushchenko and Flammarion, 2025; Huang et al., 2026), these attacks expose failure modes where a model’s safety alignment can be bypassed. Existing

jailbreak research therefore commonly evaluates attacks using attacker-centric metrics such as attack success rate (ASR), computational cost, universality, and transferability. These metrics measure how effectively and practically an attack can compromise a model from the perspective of an attacker.

For LLM safety researchers, however, this perspective is incomplete. A jailbreak attack is useful for safety research not merely because it breaks an undefended model, but because it can reveal weaknesses that will help improve a defended one (Yu et al., 2024; Mo et al., 2024; Fu et al., 2025). By shifting from *attack success* to *defense utility*, this motivates a defender-centric view of jailbreak evaluation: attacks should be valued by their downstream safety utility contribution when used as red-teaming resources in defense pipelines. Under this view, attack success is not the final objective, but an input to a broader safety-improvement process.

This shift exposes a practical decision problem. ***Which jailbreak attack(s) matter most when “matter” means improving downstream safety alignment, rather than maximizing attack success?*** This question cannot be answered by standalone attack-success metrics alone. If ASR were a reliable proxy for defense utility contribution, ranking attacks by no-defense success would already identify the most useful attacks for improving safety. However, the value of an attack may depend on the specific safety setting context, including the defense pipeline, evaluation benchmark, utility definition, and even the other attacks selected with it for defense at the same time. Thus, the setting is highly heterogeneous, and even potentially proprietary. We therefore model each setting by a black-box utility function $v_\theta(S)$, which measures the safety utility of an attack subset S under a user-specified setting θ . Our goal is not to find a universally best jailbreak attack, but to provide a setting-agnostic framework that can be instantiated wherever such subset utilities can be observed.

¹A-MESS project page: [attackshap.github.io](https://github.com/attackshap)

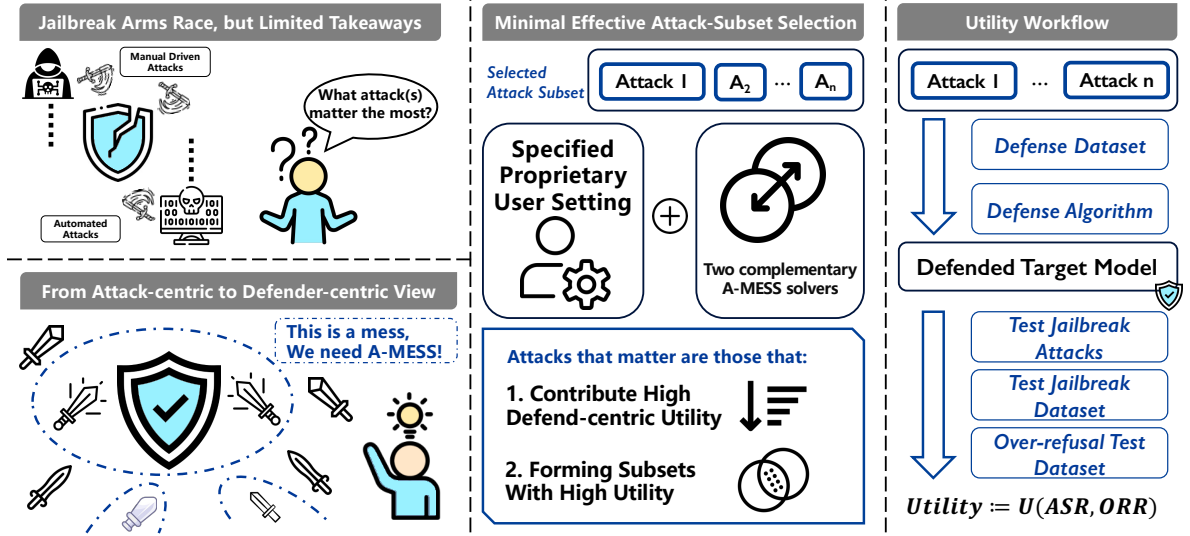


Figure 1: Conventional jailbreak research often follows an attacker-centric arms race, where attacks are primarily judged by how effectively they break a target model. A-MESS instead starts from a defender-centric question: which jailbreak attack(s) matter most for improving safety alignment under a specific, potentially proprietary user setting? Given setting-specific utility observations, A-MESS supports two complementary tasks: identifying attacks with high equitable contribution via AttackSHAP, and selecting compact attack subsets with high downstream utility.

This decision problem formulation leads to two related but distinct tasks, as illustrated in Figure 1. The first is attribution: how much credit should each attack receive for contributing to downstream utility across different contexts? We address this with **AttackSHAP**, a Shapley-based attribution score that treats jailbreak attacks as players, attack subsets S as coalitions, and $v_\theta(S)$ as the coalition value. However, directly employing exact Shapley value computation requires the true subset utility landscape, which is prohibitively expensive to obtain. Moreover, the same utility view also induces a subset-selection problem: Under a limited budget, which subset should a defender actually use? These tasks are not equivalent, since attacks with high individual contribution do not necessarily compose the highest-utility subset.

To address these tasks, we propose **A-MESS (Minimal Effective Attack-Subset Selection)**, a defender-centric framework for both attack attribution and budgeted attack-subset selection tasks. A-MESS provides two complementary solvers. A-MESS-SURROGATE leverages a surrogate model to learn an approximate utility landscape from sampled subset evaluations, enabling efficient AttackSHAP estimation and multi-budget subset search under a fixed evaluation budget. A-MESS-GREEDY directly queries true utilities and greedily selects attacks with the largest marginal gains, providing a query-efficient option when a practitioner needs a single subset under a fixed budget.

Our contributions are threefold:

- We reframe jailbreak attack evaluation from attacker-centric success to defender-centric utility, and empirically test whether ASR predicts downstream safety utility.
- We introduce **AttackSHAP**, a Shapley-based attribution score for assigning setting-specific utility contribution to individual jailbreak attacks across subset contexts.
- We formulate budgeted attack-subset selection, and propose **A-MESS (Minimal Effective Attack-Subset Selection)**, a setting-agnostic framework with greedy and surrogate-based solvers for selecting compact high-utility attack subsets from black-box utility observations. Empirical results show that A-MESS yields subset with higher defender-centric utility than attacker-centric or attribution-only selection.

2 Preliminaries and Problem Setup

We first formalize the problem discussed above. Let $\mathcal{A} = \{a_1, \dots, a_n\}$ denote the candidate attack universe, where each a_i is an attack algorithm, or equivalently attack-generated data source. As summarized in Figure 1, a safety setting θ specifies how one attack subset is converted into a defended system and how that system is evaluated. It may include the target model, defense procedure, evaluation benchmarks, computation budget, and utility definition. For any subset $S \subseteq \mathcal{A}$, we define a

defender-centric utility $v_\theta : 2^{\mathcal{A}} \rightarrow \mathbb{R}$, where $v_\theta(S)$ is the utility obtained after applying the defense workflow with attacks in S . The utility may measure jailbreak ASR reduction, or combine it with other objectives such as over-refusal penalty. A-MESS does not assume access to the internal details of θ , it only assumes that a practitioner can query $v_\theta(S)$ for selected subsets. Throughout the paper, we instantiate v_θ in both real LLM safety settings and controlled synthetic utility landscapes. In each case, utilities are computed according to the corresponding setting, and normalized to a common $[0, 1]$ scale. Detailed utility settings are provided in the experimental sections and Appendix.

In this formulation, a utility query $v_\theta(S)$ corresponds to executing the defender’s evaluation workflow for one attack subset: constructing defense data or prompts from S , applying the defense procedure, and evaluating the resulting system under the setting-specific safety metrics. This black-box view is important because many realistic safety pipelines stay proprietary, as model providers tend not to release the entire pipeline. Attacker-centric quantities such as no-defense ASR can be used as baselines, but they do not explicitly query $v_\theta(S)$.

Individual attack attribution. A central goal is to quantify how much each attack contributes to the safety utility. This cannot be captured by standalone attack metrics since they ignore subset context: Two attacks may be redundant if they expose the same failure mode, so adding one after the other yields little additional utility. Conversely, two individually modest attacks may be complementary if they cover different weaknesses and jointly improve the defended system. AttackSHAP is designed to average such marginal contributions across subset contexts. We therefore view attacks as players in a cooperative game, attack subsets $S \subseteq \mathcal{A}$ as coalitions and $v_\theta(S)$ as the coalition value. The AttackSHAP value of attack a_i is then:

$$\phi_i^\theta = \sum_{S \subseteq \mathcal{A} \setminus \{a_i\}} \frac{|S|!(n - |S| - 1)!}{n!} [v_\theta(S \cup \{a_i\}) - v_\theta(S)]. \quad (1)$$

Or equivalently, let $\Pi(\mathcal{A})$ be the set of all permutations of attacks and P_i^π be the set of attacks appearing before a_i in permutation π . The same value can be written in permutation form as

$$\phi_i^\theta = \frac{1}{n!} \sum_{\pi \in \Pi(\mathcal{A})} [v_\theta(P_i^\pi \cup \{a_i\}) - v_\theta(P_i^\pi)], \quad (2)$$

This is the Shapley value (Shapley et al., 1953; Dubey, 1975; Ghorbani and Zou, 2019) of a_i in the utility game induced by θ , and quantifies the average marginal utility of adding a_i across all possible attack orderings. We compare AttackSHAP with two boundary marginal baselines: $\text{AOO}_i^\theta = v_\theta(\{a_i\}) - v_\theta(\emptyset)$, which measures empty-context gain, and $\text{LOO}_i^\theta = v_\theta(\mathcal{A}) - v_\theta(\mathcal{A} \setminus \{a_i\})$, which measures full-context loss (Koh and Liang, 2017; Ghorbani and Zou, 2019). We compare estimated AttackSHAP against reference scores using standard regression metrics along with Pearson and Spearman correlations.

Budgeted subset selection. Attribution ranks individual attacks, but LLM safety researchers often need a subset-level decision in practice. Given a budget k , the oracle objective is to select the size- k subset with the highest safety utility

$$S_k^* = \arg \max_{S \subseteq \mathcal{A}, |S|=k} v_\theta(S). \quad (3)$$

Here S_k^* denotes the oracle optimal subset under setting θ . This fixed-size formulation captures the common case where a defender can only use k attacks for training or evaluation. In practice, finding S_k^* exactly is infeasible for large n , because it requires evaluating $\binom{n}{k}$ candidate subsets. We use S_k^* as an oracle upper bound when the full utility landscape is available, and introduce practical A-MESS solvers for approximating high-utility subsets $S_k(\text{A-MESS})$ in Section 3.

3 Method

We now describe the computational components of A-MESS. When the complete real utility landscape is too expensive to compute, we first learn a surrogate utility model $\hat{v}_\theta$ from sampled subset evaluations, enabling efficient AttackSHAP estimation. The second selects compact attack subsets under a user-specified budget, either by direct greedy search over observed true utilities or by search over the learned surrogate utility landscape.

3.1 Surrogate Utility Learning

A-MESS-SURROGATE learns an approximate utility landscape from sampled subset evaluations. For each sampled training subset $T_j \subseteq \mathcal{A}$, we represent it by a binary indicator vector $x_{T_j} \in \{0, 1\}^n$, where $(x_{T_j})_i = 1$ iff $a_i \in T_j$. Given M queried utilities, $\mathcal{D}_{\text{train}} = \{(x_{T_j}, v_\theta(T_j))\}_{j=1}^M$, we train a surrogate model $\hat{v}_\theta$ to predict subset utility from the

indicator vector by minimizing mean squared error. Once trained, $\hat{v}_\theta$ can be evaluated on unqueried subsets at low cost, enabling AttackSHAP estimation, multi-budget subset search, and repeated selection decisions without repeatedly querying the true utility function.

In practice, we split the queried subset-utility observations into training, validation, and held-out test sets. The training split is used to fit $\hat{v}_\theta$, the validation split is used for model hyper-parameter selection and early stopping, and the held-out test split evaluates whether the learned surrogate generalizes to unseen attack subsets. In synthetic landscapes, held-out subsets are sampled from the fully enumerated utility table, while in real LLM settings they correspond to additional observed subset utilities that are not used for training.

3.2 Attack Attribution and Ranking

A-MESS uses AttackSHAP to estimate the contribution of each attack. When the true utility landscape is available, we compute reference scores $\phi_i^\theta = \phi_i(v_\theta)$, either exactly in controlled synthetic settings or approximately through sampled coalitions. When using a surrogate model, A-MESS-SURROGATE estimates $\hat{\phi}_i^\theta = \phi_i(\hat{v}_\theta)$, replacing true utility evaluations with surrogate predictions. Thus, after the initial utilities are queried to fit the surrogate model, additional coalitions required by Shapley estimation are evaluated by $\hat{v}_\theta$ rather than by re-running the full defense workflow.

Attribution scores can also be used for top- k selection by ranking attacks and returning the highest-scoring k elements. We consider rankings induced by ASR, AttackSHAP, LOO (Koh and Liang, 2017), and AOO. This allows us to separate two questions: whether utility can be accurately attributed to each individual attack, and whether selecting attacks by individual importance is sufficient for subset-level utility maximization.

3.3 Budgeted Subset Selection

Beyond ranking individual attacks, A-MESS directly addresses the budgeted subset selection problem. Given a subset size k , the goal is to return a high-utility subset $S_k(m)$ under the current setting. We consider two complementary solvers.

A-MESS-GREEDY. The greedy solver directly queries v_θ and greedily selects attacks by marginal gain, making it suitable for single-budget selection when true utility queries are affordable. It starts

from $S_0 = \emptyset$ and iteratively adds the attack with the largest true marginal utility gain:

$$a_t^* = \arg \max_{a \in \mathcal{A} \setminus S_t} [v_\theta(S_t \cup \{a\}) - v_\theta(S_t)],$$

$$S_{t+1} = S_t \cup \{a_t^*\}.$$

After k steps, it returns $S_k(\text{A-MESS-GREEDY}) = S_k$. This solver is appropriate when the practitioner only needs one subset under a fixed budget and can afford adaptive true utility queries. Its query cost is $\sum_{t=0}^{k-1} (n - t) = nk - k(k-1)/2$ for a size- k subset, excluding cached evaluations.

A-MESS-SURROGATE. The surrogate solver first learns $\hat{v}_\theta$ from sampled subset evaluations and then searches in the learned utility landscape. For a target budget k , it returns

$$S_k(\text{A-MESS-SURROGATE}) = \arg \max_{S \subseteq \mathcal{A}, |S|=k} \hat{v}_\theta(S).$$

3.4 Defense-pipeline Instantiations

A-MESS only requires black-box access to $v_\theta(S)$, but in real LLM experiments we instantiate each utility query with two defense pipelines: adversarial training (Mazeika et al., 2024) and in-context defense (Wei et al., 2023b) (ICD). We distinguish defense jailbreak data from test data: the former is collected for the selected attack subset S and used to build the defended system, while the latter is held out for evaluating $v_\theta(S)$. The resulting defended system is evaluated on held-out jailbreak tests, and on over-refusal tests when included by the utility definition.

4 Experiments

4.1 Evaluated Aspects and Metrics

We evaluate A-MESS along three aspects. First, we test whether attacker-centric success is aligned with defender-centric utility by comparing the no-defense ASR ranking with the singleton utility ranking. We report Spearman’s ρ , Kendall’s τ , and representative rank shifts. Second, we evaluate whether AttackSHAP can be estimated under limited utility queries. For synthetic landscapes, we compare estimated AttackSHAP against exact references using regression and rank-correlation metrics. For real LLM utilities, where the full landscape is not exhaustively observed, we evaluate held-out prediction of $v_\theta(S)$. Third, we evaluate budgeted subset selection using raw subset utility $v_\theta(S_k(m))$ and Normalized Gain:

$$NG_k(m) = \frac{v_\theta(S_k(m)) - v_\theta(S_k(\text{BASE}))}{v_\theta(S_k^*) - v_\theta(S_k(\text{BASE}))}. \quad (4)$$

Here $S_k(\text{BASE})$ is the average random size- k subset and S_k^* is the oracle size- k subset when available. Thus, $NG_k(m)$ measures the fraction of the oracle improvement gap recovered by method m .

4.2 Experimental Setup

Synthetic utility landscapes. We use synthetic utility landscapes to model defender-side utilities over attack subsets. These experiments do not synthesize jailbreak attacks; instead, each task defines a bounded utility oracle $v(S) \in [0, 1]$ over a fixed attack universe $\mathcal{A}$. Each task contains $n = 20$ candidate attacks, allowing exact enumeration of all 2^{20} subset utilities and exact reference AttackSHAP values. We instantiate 90 utility landscapes from scalar, vector, and random generation policies. Across policies, the construction is designed to capture patterns we expect in real defense utilities, including diminishing gains, redundant attacks, and complementary attack combinations. Full construction details and an example utility-landscape visualization are provided by Figure 4 in Appendix.

Real LLM safety settings. Following the workflow in Figure 1, each real setting is specified by a target model, a defense algorithm, defense data constructed from selected attacks, and held-out test attacks, test jailbreak/over-refusal datasets used to compute defender-centric utility. We evaluate three settings with different target models and complementary goals. First, to test whether attacker-centric success predicts defender-centric utility, we use Llama-3-8B with in-context defense (Wei et al., 2023b), construct defense demonstrations from JailbreakBench (Chao et al., 2024) attacks, and compare no-defense ASR rankings with singleton defense utility evaluated on JailbreakBench and SorryBench (Xie et al., 2024). Second, to evaluate utility learning and AttackSHAP estimation, we use Qwen2.5-7B with adversarial training (Mazeika et al., 2024), where selected JailbreakBench attacks generate safety-training data and $v(S)$ is measured by the resulting robustness utility. Third, to evaluate budgeted subset selection, we use Ministral-3-8B with in-context defense; selected attacks form the defense context on JailbreakBench, while utility is computed from held-out JailbreakBench/SorryBench robustness together with over-refusal (Röttger et al., 2024). Across settings, the

attack universe $\mathcal{A}$ contains 10 attack categories instantiated on JailbreakBench and SorryBench, covering template-based (Wei et al., 2023b; Shen et al., 2024; Yoosuf et al., 2025; Zhu et al., 2024a), strategy-based (Andriushchenko and Flammarion, 2025; Huang et al., 2026; Luo et al., 2026), and learning-based (Zou et al., 2023; Zhou et al., 2025a; Zhu et al., 2024b) jailbreaks. The two defense algorithms, adversarial training and in-context defense, instantiate different ways in which an attack subset S changes the defended system. For subset selection, we evaluate budgets $k \in \{2, 4\}$. Full implementation details, attack taxonomy, and hyperparameters are provided in Appendix.

Compared methods. Table 1 summarizes the compared methods. We separate methods that produce attribution scores from methods that directly return a size- k subset. Attribution methods can be converted into subset selectors by ranking attacks and taking the top k , while selection methods optimize the subset-level decision more directly.

4.3 From Attack Success to Defense Utility

We first examine whether attacker-centric success is aligned with defender-centric utility in a real LLM safety setting. For each attack category $a \in \mathcal{A}$, we compute its no-defense ASR on JailbreakBench (Chao et al., 2024) to obtain the attacker-centric score. We then compute its singleton defense utility $v_\theta(\{a\})$ under the in-context defense setting, where the defense is constructed using only that attack category and evaluated by the corresponding defender-side safety utility. If ASR were a reliable proxy for defender-centric utility, the no-defense ASR ranking and the singleton utility ranking should be largely consistent.

However, Figure 2 shows that this is not the case. The two rankings are weakly correlated, with Spearman $\rho = 0.215$ and Kendall $\tau = 0.190$. Several attacks move substantially across the two rankings, as demonstrated in Table 2. For example, ROLEPLAY has zero no-defense ASR, yet becomes the second highest-ranked attack by singleton defense utility. In contrast, OPTIMIZED SUFFIX has the highest no-defense ASR, but drops to seventh place under defender-side utility. These results suggest that attacks that most effectively break models are not necessarily those that most effectively improve defense utility. Therefore, attacker-centric success is an incomplete criterion for identifying which attacks matter for safety alignment.

Method	Utility Budget	Description
<i>Attribution methods</i>		
ASR	0	Attacker-centric ranking by attack success rate
AOO	$n + 1$	Empty-context marginal utility gain
LOO	$n + 1$	Full-context marginal utility loss
STANDARD SHAP _v	2^n	Reference Shapley attribution from true utilities
ATTACKSHAP _{$\hat{v}$}	M	Shapley attribution from learned surrogate utilities
SHAP BASELINES	M	Standard Shapley value estimation baselines, including DataSHAP (Ghorbani and Zou, 2019), SVA (Zhang et al., 2023), SamplingSHAP@K (Kariyappa et al., 2024)
<i>Subset selection methods</i>		
RANDOM	0	Random size- k subset
TOPK($\cdot$)	score-dependent	Select top k attacks by an attribution score listed above
A-MESS-GREEDY	$nk - k(k - 1)/2$	Greedy selection using true marginal utility
A-MESS-SURROGATE	M	Search over the learned utility landscape
ORACLESEARCH _v	$\binom{n}{k}$	Exhaustive true-utility search; not feasible and acts as upper bound only

Table 1: Compared methods. Utility budget denotes the number of true subset-utility evaluations $v(S)$, excluding attacker-centric quantities such as ASR. M is the utility budget specified by each concrete method.

Attack	ASR Rank	No-defense ASR	Utility Rank	$v_\theta(\{a\})$	ΔRank
Optimized Suffix	1	0.620	7	0.846	-6
Refusal Suppression	2	0.270	1	0.904	+1
Roleplay	9	0.000	2	0.890	+7
Spearman $\rho = 0.215$, Kendall $\tau = 0.190$.					

Table 2: Representative ranking shifts between attacker-centric success and defender-centric utility. ASR Rank is computed from no-defense attack success rate, while Utility Rank is computed from singleton in-context-defense utility. We define $\Delta\text{Rank} = \text{ASR Rank} - \text{Utility Rank}$, so positive values indicate that an attack ranks higher under defender-side utility than under attack success.

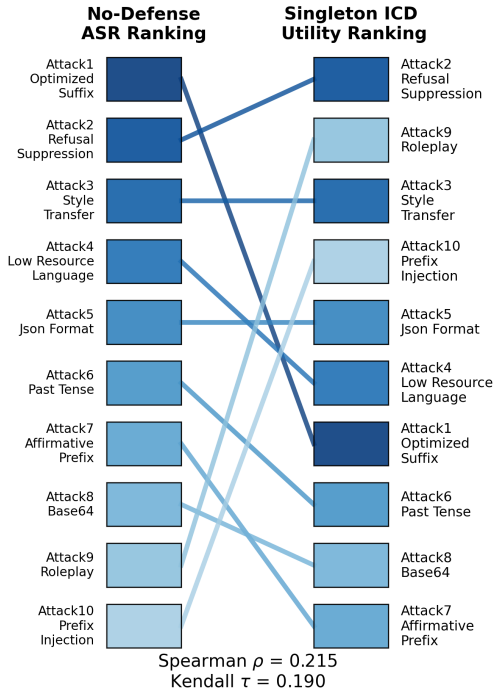


Figure 2: Attack ASR ranking differs from defender-centric utility ranking. Attack categories are ordered by no-defense ASR on the left and by singleton in-context-defense utility $v_\theta(\{a\})$ on the right. Each line connects the same attack across two rankings. The low rank correlation, Spearman $\rho = 0.215$ and Kendall $\tau = 0.190$, shows that attacker-centric success is an incomplete proxy for defender-centric utility in this setting.

$n_{\text{train}}:n_{\text{test}}$	MAE $\downarrow$	$R^2 \uparrow$	Pearson $\uparrow$	Spearman $\uparrow$
50:250	0.078 $\pm$ 0.005	0.779 $\pm$ 0.031	0.888 $\pm$ 0.017	0.893 $\pm$ 0.019
100:200	0.071 $\pm$ 0.003	0.817 $\pm$ 0.018	0.908 $\pm$ 0.008	0.914 $\pm$ 0.009
150:150	0.066 $\pm$ 0.004	0.828 $\pm$ 0.026	0.912 $\pm$ 0.015	0.917 $\pm$ 0.016
200:100	0.067 $\pm$ 0.006	0.825 $\pm$ 0.038	0.912 $\pm$ 0.021	0.917 $\pm$ 0.020

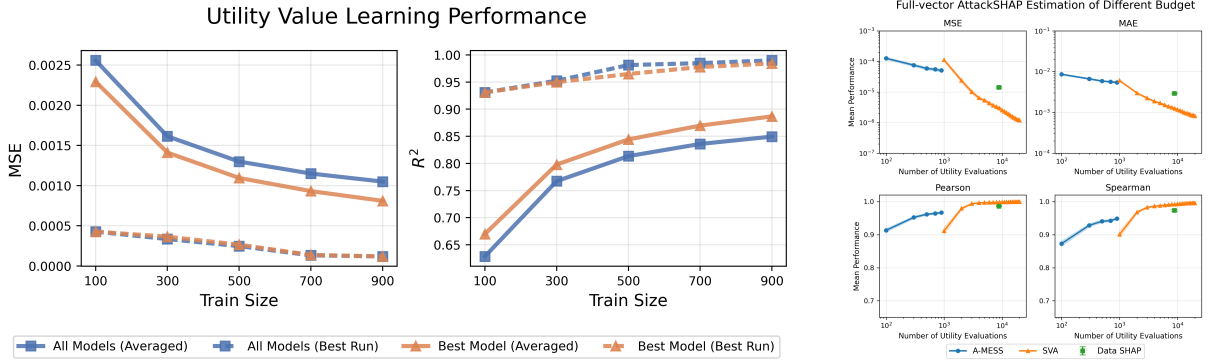
Table 3: Real-LLM subset utility prediction. We train the A-MESS-SURROGATE on observed subset utilities and evaluate held-out prediction of $v(S)$. Metrics are mean $\pm$ std across random splits.

Method	$k = 1$	$k = 3$	$k = 5$	$k = 7$	$k = 9$
<i>Attribution Baselines</i>					
TOPK-AOO _v	100.00	53.12	46.66	37.76	31.52
TOPK-LOO _v	33.99	36.48	44.79	57.33	63.80
TOPK- ϕ_v	65.39	61.10	67.03	68.51	69.28
TOPK- ϕ	59.66	57.37	62.94	65.01	71.07
<i>A-MESS Solvers</i>					
A-MESS-GREEDY	100.00	85.47	83.36	83.20	82.69
A-MESS-SURROGATE	59.78	73.18	74.48	79.79	84.52

Table 4: Synthetic subset selection results. We report Normalized Gain NG_k (%) averaged over 90 utility landscapes, where higher values indicate closer recovery of the oracle subset utility.

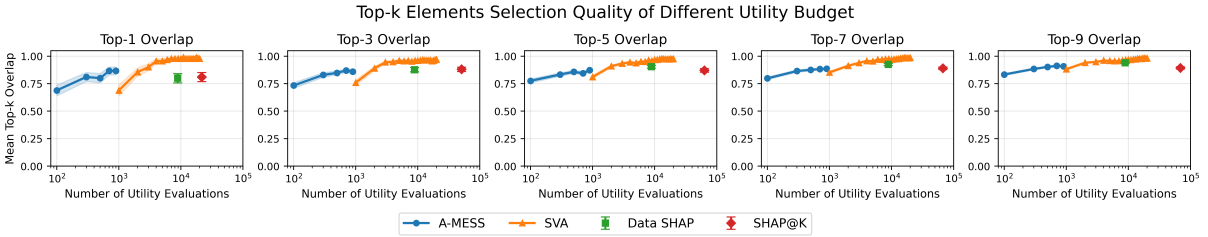
4.4 Budgeted AttackSHAP Estimation

This subsection evaluates A-MESS-Surrogate at two levels. On synthetic landscapes, where the full utility landscape and reference AttackSHAP is available, we test whether the surrogate recovers both the utility function and the resulting AttackSHAP values. Existing Shapley estimators are compared. On real LLM utilities, where exhaustive evaluation is infeasible, we test held-out prediction of $v_\theta(S)$ as the observable proxy.



(a) Surrogate utility learning performance. We evaluate how accurately the learned surrogate utility model approximates the ground-truth subset utility under different numbers of sampled utility evaluations. The surrogate model increasingly approximates the underlying subset utility landscape as more sampled subset evaluations are provided. R^2 achieves more than 0.9 for the best performing surrogate models.

(b) Full-vector AttackSHAP estimation. We compare AttackSHAP estimation quality under different utility budgets using error-based and correlation-based metrics.



(c) Top- k attack attribution quality. We evaluate whether estimated attribution scores recover the top- k reference attacks.

Figure 3: Synthetic evaluation of A-MESS-SURROGATE. Panel 3a evaluates surrogate utility prediction under increasing utility-query budgets. Panel 3b compares surrogate-based AttackSHAP estimates with reference full-vector AttackSHAP values. Panel 3c measures top- k overlap with the attacks identified by reference AttackSHAP.

We first evaluate whether A-MESS-SURROGATE can estimate AttackSHAP with limited utility queries. On synthetic landscapes, the full 2^{20} utility table provides oracle AttackSHAP references. Figure 3 shows that the learned utility model reaches high R^2 in the high-budget regime. More importantly, the AttackSHAP estimates match or outperform strong existing Shapley estimators on both full-vector ϕ prediction and top- k coverage, while requiring only hundreds of true utility queries rather than thousands or tens of thousands. A-MESS reaches a strong accuracy efficiency tradeoff for standard Shapley estimation.

We further evaluate the same surrogate on real Qwen2.5-7B adversarial-training utilities. Since the full real utility landscape cannot be exhaustively observed, we test held-out prediction of $v(S)$, reporting mean and standard deviation over five random train/test splits for each training-set size. Table 3 shows that the surrogate already obtains strong utility prediction performance with 150 training subsets, and the improvement from 150 to 200 training subsets is small, suggesting convergence. This indicates that, in this real LLM setting, roughly 200 observed subset utilities are sufficient for A-MESS to learn a practical approximation of the defender-centric utility landscape.

4.5 Minimal Effective Subset Selection

We finally evaluate whether attack attribution directly yields the best subset, and whether A-MESS can improve subset-level utility. Table 4 reports Normalized Gain on 90 synthetic utility landscapes. Attribution-based top- k selection improves over random selection, but remains clearly below direct subset optimization. Among attribution baselines, TOPK- ϕ_v and TOPK- $\hat{\phi}$ are the strongest, reaching roughly 60–71% Normalized Gain for nontrivial budgets. In contrast, A-MESS-GREEDY recovers at least 82.69% of the oracle improvement gap for $k > 1$, while A-MESS-SURROGATE improves as the budget grows and reaches 84.52% at $k = 9$. These results show that ranking attacks by individual contribution is not sufficient, and explicitly optimizing the subset can recover more oracle gain.

Table 5 evaluates the same question in the real ICD setting on Ministral-3. A-MESS selects the highest-utility subset under both evaluated budgets. Both solvers choose $\{1, 2\}$ for $k = 2$, reaching top utility 0.8539, and $\{1, 2, 3, 6\}$ for $k = 4$, reaching top utility 0.9397. This trend is consistent with the synthetic results, that high-utility attack subsets are not necessarily recovered by attacker-centric metrics or individual attribution.

Method	Selected subset	Utility $\uparrow$	$\Delta U\uparrow$	Method	Selected subset	Utility $\uparrow$	$\Delta U\uparrow$
NO DEFENSE	–	0.1960	0.0000	NO DEFENSE	–	0.1960	0.0000
<i>Baselines with budget $k = 2$</i>				<i>Baselines with budget $k = 4$</i>			
RANDOM	Averaged over 5 random subsets	0.6183	0.4223	RANDOM	Averaged over 5 random subsets	0.5342	0.3382
TOPK-ASR	Attack ₁ , Attack ₈	0.5716	0.3756	TOPK-ASR	Attack ₁ , Attack ₂ , Attack ₃ , Attack ₈	0.8613	0.6653
TOPK-AOO	Attack ₂ , Attack ₄	0.5379	0.3419	TOPK-AOO	Attack ₀ , Attack ₂ , Attack ₄ , Attack ₅	0.5448	0.3488
TOPK-LOO	Attack ₂ , Attack ₈	0.6447	0.4487	TOPK-LOO	Attack ₂ , Attack ₇ , Attack ₈ , Attack ₉	0.5616	0.3656
<i>A-MESS Solvers with budget $k = 2$</i>				<i>A-MESS Solvers with budget $k = 4$</i>			
A-MESS-GREEDY	Attack ₁ , Attack ₂	0.8539	0.6579	A-MESS-GREEDY	Attack ₁ , Attack ₂ , Attack ₃ , Attack ₆	0.9397	0.7437
A-MESS-SURROGATE	Attack ₁ , Attack ₂	0.8539	0.6579	A-MESS-SURROGATE	Attack ₁ , Attack ₂ , Attack ₃ , Attack ₆	0.9397	0.7437

Table 5: Real in-context-defense subset selection results on Ministral-3. Utility is defined as $U = 0.8 * \text{Score}_{\text{ASR}} + 0.2 * \text{Score}_{\text{ORR}}$, where $\text{Score}_{\text{ORR}} = 1 - \text{ORR}$ denotes over-refusal rate on XSTest. ΔU denotes the absolute utility gain over the no-defense baseline. Both A-MESS solvers select the highest-utility subset under both k budget.

5 Discussion

The main contribution of this work is not merely a numerical improvement, but a shift in how jailbreak attacks should be interpreted in safety research. Much of jailbreak research has been driven by an arms race. However, a first-principles view of doing research should start from the very final goal (Lu et al., 2025). Applied to jailbreak research, this view suggests that attacks should be valued not only by how often they break an undefended model, but by how much they improve downstream safety when used as red-teaming resources. Our results make this distinction concrete: ASR and singleton defense utility are weakly aligned in our real setting, and attacks with high individual attribution do not necessarily form the best subset. This reframes jailbreak attacks as defense resources whose value depends on the downstream safety objective.

This perspective also motivates the black-box, setting-agnostic design of A-MESS. Safety settings may be largely heterogeneous and proprietary, and many production pipelines are only partially observable. Rather than seeking a universally best subset, as a setting-agnostic framework A-MESS requires only utility observations $v_\theta(S)$, allowing practitioners to instantiate the framework under their own safety objectives. This does not solve the broader transparency gap, but it provides a practical interface for connecting jailbreak evaluation to measurable safety improvement.

6 Related Work

Jailbreak attacks and safety defenses. Prior work has proposed diverse jailbreak attacks against aligned LLMs, including template-based (Wei et al., 2023b; Shen et al., 2024; Yoosuf et al., 2025; Zhu et al., 2024a), strategy-based (Andriushchenko and Flammarion, 2025; Huang et al., 2026; Luo et al., 2026), and learning-based attacks (Zou et al.,

2023; Zhou et al., 2025a; Zhu et al., 2024b). These attacks are typically evaluated by attacker-centric metrics such as ASR, transferability, universality, and cost. In parallel, LLM safety work has studied refusal training, in-context defenses, guardrails, and over-refusal-aware evaluation (Yu et al., 2024; Mo et al., 2024; Fu et al., 2025; Röttger et al., 2024; Zhang et al., 2025; Cui et al., 2025; Pan et al., 2025). These defenses show that attack data can improve safety, but rarely ask which attacks are most useful for downstream safety alignment. A-MESS bridges this gap by treating jailbreak attacks as red-teaming resources and evaluating them through setting-specific defender-side utility.

Game-theoretic valuation and subset selection.

Shapley values have been widely used to attribute machine-learning utility to data points, features, or training examples (Shapley et al., 1953; Dubey, 1975; Ghorbani and Zou, 2019), with later work studying more efficient Shapley approximation and top- k identification (Zhang et al., 2023; Kariyappa et al., 2024). A-MESS differs in both the object being valued and the decision it supports: the players are jailbreak attacks, the coalition value is downstream defender-side utility under a safety pipeline, and attribution is paired with budgeted subset selection rather than treated as the final goal.

7 Conclusion

We presented A-MESS, a defender-centric framework for attributing and selecting jailbreak attacks by their contribution to downstream safety utility. Across controlled utility landscapes and real settings, our results show that high-ASR attacks need not yield high defender-side utility, and that high-contribution attacks do not necessarily form the best subset. These findings suggest that jailbreak evaluation should go beyond attack success alone and ask which attacks help build safer models.

Limitations

A-MESS assumes that the defender-side utility of an attack subset is meaningful, which requires the defense procedure to be coupled with the selected attacks. This includes settings such as adversarial training, in-context defense, or other pipelines where different attack subsets can lead to different defended systems. The framework is less suitable for defenses whose behavior is independent of the selected attack set. In such cases (Xie et al., 2023), $v_\theta(S)$ may not meaningfully vary with S , making attack attribution or subset selection ill-posed.

LLM safety alignment also includes downstream concerns beyond jailbreak robustness, such as backdoor robustness (Yang et al., 2024), watermarking (Dathathri et al., 2024), hallucination (Ji et al., 2023), and broader reliability evaluation (Zhou et al., 2025b). Since A-MESS only requires a setting-specific subset utility $v_\theta(S)$, the framework is in principle applicable to these settings as well. Due to space and computation constraints, however, this paper focuses on jailbreak attacks as the primary downstream task, leaving broader safety-alignment instantiations to future work.

References

- Maksym Andriushchenko and Nicolas Flammarion. 2025. [Does refusal training in LLMs generalize to the past tense?](#) In *The Thirteenth International Conference on Learning Representations*.
- Tim Beyer, Yan Scholten, Leo Schwinn, and Stephan Günnemann. 2026. [Sampling-aware adversarial attacks against large language models](#). In *The Fourteenth International Conference on Learning Representations*.
- Nicholas Carlini, Milad Nasr, Christopher A Choquette-Choo, Matthew Jagielski, Irena Gao, Pang Wei Koh, Daphne Ippolito, Florian Tramèr, and Ludwig Schmidt. 2023. Are aligned neural networks adversarially aligned? *Advances in Neural Information Processing Systems*, 36:61478–61500.
- Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J Pappas, Florian Tramèr, and 1 others. 2024. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. *arXiv preprint arXiv:2404.01318*.
- Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. 2025. Jailbreaking black box large language models in twenty queries. In *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 23–42. IEEE.
- Junjie Chu, Yugeng Liu, Ziqing Yang, Xinyue Shen, Michael Backes, and Yang Zhang. 2025. Jailbreakradar: Comprehensive assessment of jailbreak attacks against llms. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21538–21566.
- Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. 2025. [OR-bench: An over-refusal benchmark for large language models](#). In *Forty-second International Conference on Machine Learning*.
- Sumanth Dathathri, Abigail See, Sumedh Ghaisas, Po-Sen Huang, Rob McAdam, Johannes Welbl, Vandana Bachani, Alex Kaskasoli, Robert Stanforth, Tatiana Matejovicova, and 1 others. 2024. Scalable watermarking for identifying large language model outputs. *Nature*, 634(8035):818–823.
- Pradeep Dubey. 1975. On the uniqueness of the shapley value. *International Journal of Game Theory*, 4(3):131–139.
- Shaopeng Fu, Liang Ding, Jingfeng Zhang, and Di Wang. 2025. Short-length adversarial training helps llms defend long-length jailbreak attacks: Theoretical and empirical evidence. *arXiv preprint arXiv:2502.04204*.
- Amirata Ghorbani and James Zou. 2019. Data shapley: Equitable valuation of data for machine learning. In *International conference on machine learning*, pages 2242–2251. PMLR.
- Xun Huang, Simeng Qin, Xiaoshuang Jia, Ranjie Duan, Huanqian Yan, Zhitao Zeng, Fei Yang, Yang Liu, and Xiaojun Jia. 2026. [Obscure but effective: Classical chinese jailbreak prompt optimization via bio-inspired search](#). In *The Fourteenth International Conference on Learning Representations*.
- Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614*.
- Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating llm hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 1827–1843.
- Sanjay Kariyappa, Leonidas Tsepenekas, Freddy Lécué, and Daniele Magazzeni. 2024. Shap@ k: Efficient and probably approximately correct (pac) identification of top-k features. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 13068–13075.

- Pang Wei Koh and Percy Liang. 2017. Understanding black-box predictions via influence functions. In *International conference on machine learning*, pages 1885–1894. PMLR.
- Zeyi Liao and Huan Sun. 2024. [AmpleGCG: Learning a universal and transferable generative model of adversarial suffixes for jailbreaking both open and closed LLMs](#). In *First Conference on Language Modeling*.
- Renzhi Lu, Zonghe Shao, Yuemin Ding, Ruijuan Chen, Dongrui Wu, Housheng Su, Tao Yang, Fumin Zhang, Jun Wang, Yang Shi, and 1 others. 2025. Discovery of the reward function for embodied reinforcement learning agents. *Nature Communications*, 16(1):11064.
- Xuan Luo, Yue Wang, Zefeng He, Geng Tu, Jing Li, and Ruifeng Xu. 2026. A simple and efficient learning-style prompting for llm jailbreaking. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 2389–2406.
- Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhae, Nathaniel Li, Steven Basart, Bo Li, and 1 others. 2024. Harm-bench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*.
- Yichuan Mo, Yuji Wang, Zeming Wei, and Yisen Wang. 2024. [Fight back against jailbreaking via prompt adversarial tuning](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Licheng Pan, Yongqi Tong, Xin Zhang, Xiaolu Zhang, Jun Zhou, and Zhixuan Chu. 2025. Understanding and mitigating overrefusal in llms from an unveiling perspective of safety decision boundary. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 21068–21086.
- Qibing Ren, Hao Li, Dongrui Liu, Zhanxu Xie, Xiaoya Lu, Yu Qiao, Lei Sha, Junchi Yan, Lizhuang Ma, and Jing Shao. 2025. LLMs know their vulnerabilities: Uncover safety gaps through natural distribution shifts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24763–24785.
- Paul Röttger, Hannah Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. 2024. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5377–5400.
- Lloyd S Shapley and 1 others. 1953. A value for n-person games.
- Xinyue Shen, Zeyuan Chen, Michael Backes, Yun Shen, and Yang Zhang. 2024. "do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1671–1685.
- Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023a. Jailbroken: How does llm safety training fail? *Advances in Neural Information Processing Systems*, 36:80079–80110.
- Zeming Wei, Yifei Wang, Ang Li, Yichuan Mo, and Yisen Wang. 2023b. Jailbreak and guard aligned language models with only few in-context demonstrations. *arXiv preprint arXiv:2310.06387*.
- Tinghao Xie, Xiangyu Qi, Yi Zeng, Yangsibo Huang, Udari Madhushani Schwag, Kaixuan Huang, Luxi He, Boyi Wei, Dacheng Li, Ying Sheng, and 1 others. 2024. Sorry-bench: Systematically evaluating large language model safety refusal behaviors. *arXiv preprint arXiv:2406.14598*.
- Yueqi Xie, Jingwei Yi, Jiawei Shao, Justin Curl, Lingjuan Lyu, Qifeng Chen, Xing Xie, and Fangzhao Wu. 2023. Defending chatgpt against jailbreak attack via self-reminders. *Nature Machine Intelligence*, 5(12):1486–1496.
- Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. 2024. Watch out for your agents! investigating backdoor threats to llm-based agents. *Advances in Neural Information Processing Systems*, 37:100938–100964.
- Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaying Song, Ke Xu, and Qi Li. 2024. Jailbreak attacks and defenses against large language models: A survey. *arXiv preprint arXiv:2407.04295*.
- Shehel Yoosuf, Temoor Ali, Ahmed Lekssays, Mashael AlSabah, and Issa Khalil. 2025. Structtransform: A scalable attack surface for safety-aligned large language models. In *European Symposium on Research in Computer Security*, pages 488–507. Springer.
- Lei Yu, Virginie Do, Karen Hambardzumyan, and Nicola Cancedda. 2024. Robust llm safeguarding via refusal feature adversarial training. *arXiv preprint arXiv:2409.20089*.
- Jiayao Zhang, Qiheng Sun, Jinfei Liu, Li Xiong, Jian Pei, and Kui Ren. 2023. Efficient sampling approaches to shapley value approximation. *Proceedings of the ACM on Management of Data*, 1(1):1–24.
- Yuyou Zhang, Miao Li, William Han, Yihang Yao, Zhepeng Cen, and Ding Zhao. 2025. Safety is not only about refusal: Reasoning-enhanced fine-tuning for interpretable llm safety. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 18727–18746.
- Yukai Zhou, Jian Lou, Zhijie Huang, Zhan Qin, Sibe Yang, and Wenjie Wang. 2025a. Don't say no: Jailbreaking llm by suppressing refusal. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 25224–25249.

- Yukai Zhou, Sibe Yang, and Wenjie Wang. 2025b. Beyond jailbreaks: Revealing stealthier and broader llm security risks stemming from alignment failures. *arXiv preprint arXiv:2506.07402*.
- Sicheng Zhu, Brandon Amos, Yuandong Tian, Chuan Guo, and Ivan Evtimov. 2024a. Advprefix: An objective for nuanced llm jailbreaks. *arXiv preprint arXiv:2412.10321*.
- Sicheng Zhu, Ruiyi Zhang, Bang An, Gang Wu, Joe Barrow, Zichao Wang, Furong Huang, Ani Nenkova, and Tong Sun. 2024b. [AutoDAN: Interpretable gradient-based adversarial attacks on large language models](#). In *First Conference on Language Modeling*.
- Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*.

A Appendix Overview

This appendix provides additional details for the experimental settings used in the main paper. Appendix B describes the construction of synthetic utility landscapes and the utility-landscape visualization. Appendix C reports raw synthetic subset utilities corresponding to the Normalized Gain results in the main paper. Appendix D summarizes the real LLM safety settings. Appendix E lists the attack categories used in the real LLM experiments. Appendix F gives implementation details for baselines and A-MESS solvers.

B Synthetic Utility Landscapes

Purpose. The synthetic experiments are designed to evaluate A-MESS under controlled defender-side utility functions where the full subset utility table is observable. They do not synthesize jailbreak prompts. Instead, each synthetic task defines a set function $v(S)$ over subsets $S \subseteq \mathcal{A}$, which allows us to compute exact AttackSHAP values, oracle size- k subsets, and Normalized Gain.

Task construction. Each synthetic task contains $n = 20$ candidate attacks, so the full utility landscape contains 2^{20} subset values. We instantiate 90 landscapes using three generation policies: scalar, vector, and random. Each policy contributes 30 landscapes with different random seeds. All utilities are min-max normalized within each landscape to the range $[0, 1]$.

Generation policies. The scalar policy assigns each attack a latent scalar strength and aggregates the selected strengths. The vector policy assigns each attack a vector in a latent coverage space and scores the subset by the coverage induced by the selected vectors. The random policy starts from per-attack base potentials and adds deterministic subset-level variation. Across all policies, the latent score is further shaped by shared mechanisms that mimic patterns expected in real defense utilities: diminishing gains as subsets grow, a weak diversity bonus for covering more synthetic attack families, and pairwise interactions that can model either redundancy or complementarity between attacks.

Exact references. For each landscape, we enumerate all 2^{20} subset utilities and compute exact AttackSHAP values from the full table. These exact values serve as references for evaluating surro-

gate utility learning, full-vector AttackSHAP estimation, top- k attribution recovery, and budgeted subset selection. For surrogate learning, sampled subset utilities are split into training and validation subsets, and held-out utility prediction is evaluated on unqueried subsets from the same landscape.

Utility-landscape visualization. Figure 4 visualizes one representative synthetic landscape. It shows how subset utility varies across subset sizes and illustrates that neighboring subsets can have different marginal gains even when their sizes are similar. This provides an intuitive view of why subset-level utility cannot always be recovered by ranking attacks independently.

C Additional Synthetic Subset-Selection Results

The main text reports Normalized Gain for synthetic subset selection because it normalizes each utility landscape by its own random-baseline and oracle gap before averaging. For completeness, Table 6 reports the corresponding raw subset utility $v_\theta(S_k(m))$, averaged over the same 90 minmax utility landscapes used in Table 4. Because Normalized Gain is averaged per landscape, it is not obtained by normalizing the marginal means in Table 6. The raw utilities show the same qualitative pattern as Normalized Gain: direct subset optimization improves over attribution-only top- k selection, while ORACLESEARCH_v remains the upper bound.

D Real LLM Safety Settings

Table 7 summarizes the three real LLM safety settings used in the main experiments.

Defense data versus test data. In all real LLM settings, defense data and test data are separated. Defense data are generated from the selected attack subset S and are used either to construct in-context demonstrations or to create adversarial-training examples. Test data are held out and are used only to compute the final utility $v_\theta(S)$. For in-context defense, selected attacks change the defended prompt at inference time. For adversarial training, selected attacks change the fine-tuning data and therefore the resulting defended model.

Utility definitions. For robustness-only settings, utility is based on the post-defense reduction of jailbreak ASR. For the subset-selection setting, SCORE_{ASR} is the relative balanced-ASR gain over

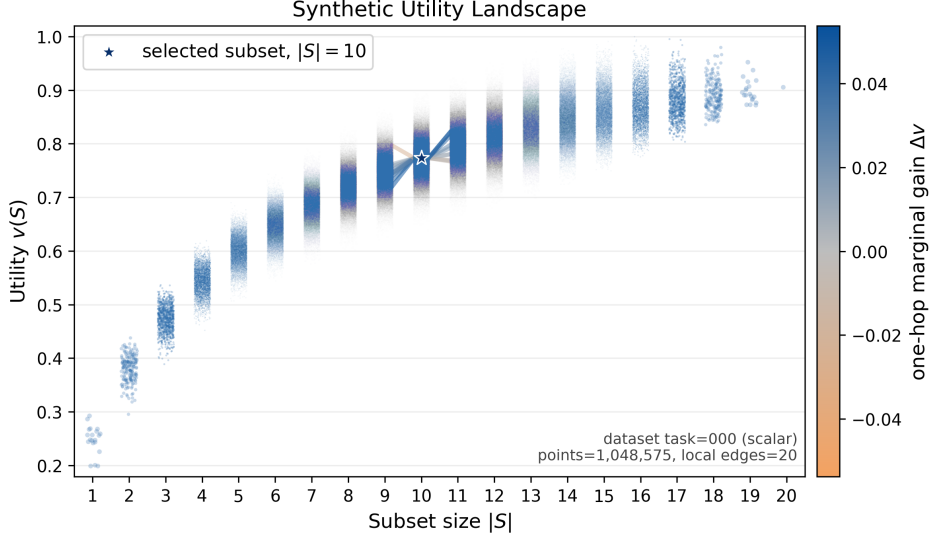


Figure 4: Example synthetic utility landscape. Each point corresponds to one attack subset, grouped by subset size $|S|$ on the x-axis and plotted by utility $v(S)$ on the y-axis. The highlighted star marks one selected subset, and local edges show one-hop neighboring subsets obtained by adding or removing one attack. Edge color indicates the one-hop marginal utility change Δv .

Method	$k = 1$	$k = 3$	$k = 5$	$k = 7$	$k = 9$
<i>Natural Baseline</i>					
RANDOM	0.2941	0.4527	0.5479	0.6171	0.6663
<i>Attribution Baselines</i>					
TOPK-AOO _v	0.4456	0.5580	0.6471	0.6980	0.7329
TOPK-LOO _v	0.3497	0.5208	0.6344	0.7344	0.8055
TOPK- ϕ_v	0.4018	0.5641	0.6814	0.7592	0.8158
TOPK- $\hat{\phi}$	0.3828	0.5577	0.6682	0.7506	0.8203
<i>A-MESS Solvers</i>					
A-MESS-GREEDY	0.4456	0.6119	0.7143	0.7897	0.8449
A-MESS-SURROGATE	0.3791	0.5833	0.6958	0.7841	0.8527
<i>Oracle Upper Bound</i>					
ORACLESEARCH _v	0.4456	0.6365	0.7450	0.8253	0.8851

Table 6: Raw subset utility averaged over the same 90 synthetic utility landscapes. For each method m , $S_k(m)$ denotes the selected size- k subset, and each entry reports $v_\theta(S_k(m))$. These marginal means are reported for interpretability, and the Normalized Gain in Table 4 is computed per landscape before averaging.

the no-defense baseline, clipped to $[0, 1]$, and $\text{SCORE}_{\text{ORR}} = 1 - \text{ORR}$, where ORR is the over-refusal rate on XSTest. This utility rewards jail-break robustness while penalizing defenses that over-refuse benign requests.

E Attack Taxonomy

Table 8 lists the ten attack categories used across the real LLM experiments.

F Baseline and Solver Details

Attribution baselines. ASR ranks attacks by no-defense attack success rate and does not query the defender-side utility function. AOO measures

empty-context marginal gain,

$$\text{AOO}_i = v_\theta(\{a_i\}) - v_\theta(\emptyset),$$

while LOO measures full-context marginal loss,

$$\text{LOO}_i = v_\theta(\mathcal{A}) - v_\theta(\mathcal{A} \setminus \{a_i\}).$$

AttackSHAP averages marginal utility over all subset contexts. In synthetic experiments, reference AttackSHAP is computed exactly from the full utility table. In surrogate-based estimation, AttackSHAP is computed by replacing true utility queries with $\hat{v}_\theta$.

Subset-selection baselines. For budget k , RANDOM averages over random size- k subsets. TOPK-

Table 7: Real LLM safety settings used in the main experiments.

Setting	Target model	Defense pipeline	Defense / test data	Utility and role in paper
ASR–utility alignment	Llama-3-8B	In-context defense	Defense demonstrations are constructed from singleton JailbreakBench attack categories; held-out JailbreakBench and SorryBench attacks are used for safety evaluation.	Compares no-defense ASR ranking with singleton defender-side utility ranking to test whether attacker-centric success is a reliable proxy.
AttackSHAP estimation	Qwen2.5-7B	Adversarial training	Selected JailbreakBench attacks generate safety-training data; held-out JailbreakBench attacks are used to measure post-defense robustness.	Provides real subset-utility observations for learning $\hat{v}_\theta$; held-out prediction of $v_\theta(S)$ evaluates whether the surrogate captures the real utility landscape.
Subset selection	Ministral-3-8B	In-context defense	Selected JailbreakBench attacks form the defense context; held-out JailbreakBench and SorryBench attacks evaluate jailbreak robustness, and XSTest evaluates over-refusal.	Evaluates size- k subset selection with $k \in \{2, 4\}$, using $U = 0.8 \text{Score}_{\text{ASR}} + 0.2 \text{Score}_{\text{ORR}}$.

ASR, TOPK-AOO, TOPK-LOO, and TOPK-ATTACKSHAP rank individual attacks by the corresponding score and return the top k . A-MESS-GREEDY adaptively queries true utilities and adds the attack with the largest observed marginal gain at each step. A-MESS-SURROGATE first learns $\hat{v}_\theta$ from sampled subset utilities and then searches the learned utility landscape. When the full synthetic landscape is available, ORACLESEARCH exhaustively searches all size- k subsets and is used only as an upper bound.

Synthetic surrogate training. In the synthetic experiments, A-MESS-Surrogate is trained under utility-query budgets $M \in \{100, 300, 500, 700, 900\}$. For each task and budget, models are selected by validation error, and the selected surrogate is used for downstream AttackSHAP estimation and subset search. Because the synthetic utility table is fully enumerated, held-out evaluation can be performed on subsets that were never used for surrogate training.

Table 8: Attack categories used in the real LLM experiments.

Attack name	Brief description
Base64 (Chu et al., 2025)	Encodes the harmful request and asks the model to decode or operate on the encoded content.
Roleplay (Shen et al., 2024)	Frames the request through a persona, fictional role, or simulated scenario to weaken refusal behavior.
Prefix Injection (Wei et al., 2023a)	Prepends instructions that attempt to override the model’s safety policy or system behavior.
Refusal Suppression (Wei et al., 2023a)	Explicitly asks the model not to refuse, apologize, moralize, or mention safety constraints.
JSON Format (Yoosuf et al., 2025)	Requests the answer in a structured format, such as JSON fields, to reduce natural-language refusal cues.
Affirmative Prefix (Zhu et al., 2024a)	Starts the assistant response with an affirmative phrase so generation continues in a compliant direction.
Past Tense (Andriushchenko and Flammarion, 2025)	Rewrites the harmful instruction as a past-tense or historical description.
Low-resource Language (Huang et al., 2026)	Translates or obfuscates the request through a lower-resource language or multilingual form.
Style Transfer (Luo et al., 2026)	Asks the model to express the harmful content in a specific style, genre, or transformed surface form.
Optimized Suffix (Zou et al., 2023; Zhou et al., 2025a; Zhu et al., 2024b)	Appends an optimized suffix or model-specific continuation pattern to induce harmful compliance.